

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

Dr

QUESTIONS Marks: 50			Total
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5

5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D. Gnanadeepthi.

ANSWERS

1. Query processing in a DBMS is the process of converting a user's SQL query into an efficient sequence of operations that retrieves the required data. The main steps are:

1. Parsing and Translation:

The DBMS checks the SQL query for syntax and semantic errors. It verifies table names, column names, data types, and user permissions. The query is then converted into an internal representation such as a query tree or relational algebra expression.

2. Query Optimization:

The optimizer examines different possible ways to execute the query. For example, it can choose different join orders, indexes, join algorithms, or access paths.

3. Query Evaluation and Execution:

The selected execution plan is passed to the query execution engine. The DBMS performs operations such as selection, projection, join, sorting, and aggregation to produce the final result.

The query optimizer decides the best execution plan by comparing alternative plans. In cost-based optimization, it estimates the cost using factors such as the number of disk I/O operations, CPU time, memory usage, table size, number of tuples, available indexes, and data distribution. The plan with the lowest estimated cost is generally selected.

For example, if a query searches for one student using an indexed primary key, the optimizer may choose an index scan instead of scanning the entire STUDENT table.

2. Heuristic optimization uses predefined rules to improve a query plan. It does not calculate the exact execution cost of every possible plan. Common heuristic rules include:

- Perform selection operations as early as possible.
- Perform projection early to reduce unnecessary columns.
- Execute the most restrictive conditions first.
- Join smaller relations before larger relations.

It is fast and simple but may not always produce the optimal execution plan.

Cost-based optimization generates and compares different execution plans. It estimates the cost of each plan based on information such as table size, number of records, indexes, data

distribution, disk I/O, CPU usage, and memory requirements. The plan with the lowest estimated cost is selected.

Heuristic Optimization

Uses predefined rules

Faster

Does not require detailed statistics

May not find the best plan

Simple to implement

Cost-Based Optimization

Uses estimated execution cost

More computationally expensive

Requires database statistics

Usually produces a better plan

More complex

When to use:

Heuristic optimization is useful for simple queries or when fast optimization is required.

Cost-based optimization is preferred for complex queries involving large tables, multiple joins, and different indexes because choosing an efficient plan can significantly improve performance.

3. In a real project, database connectivity is implemented using a database driver and an appropriate connectivity API. The general steps are:

1. Install and configure the required database driver.
2. Store database URL, username, and password securely.
3. Establish a connection with the database.
4. Create statements or prepared statements.
5. Execute SQL queries or updates.
6. Process the returned results.
7. Handle exceptions and errors.
8. Close database resources after use.

For example, in a Java application, JDBC can be used to connect to MySQL:

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/student_db",  
    "root",  
    "password"  
);
```

Prepared statements should be used instead of directly concatenating user input into SQL queries because they improve security and help prevent SQL injection.

Difference between JDBC and ODBC

JDBC	ODBC
Stands for Java Database Connectivity	Stands for Open Database Connectivity
Mainly designed for Java applications	Language-independent database access standard
Uses Java drivers	Uses ODBC drivers
Platform-independent with Java	Depends on ODBC driver support
Suitable for Java applications	Suitable for applications written in different languages

JDBC is normally used when developing Java applications, while ODBC is useful when a common database connectivity interface is needed across different programming languages and applications.

4. ACID properties ensure that database transactions are processed reliably and correctly.

A – Atomicity:

Atomicity means a transaction is treated as one complete unit. Either all operations are completed, or none of them are performed.

Example: In a bank transfer of ₹5,000, the amount must be deducted from Account A and added to Account B. If the second operation fails, the deduction from Account A must also be cancelled.

C – Consistency:

Consistency ensures that a transaction moves the database from one valid state to another valid state while maintaining all rules and constraints.

Example: In an online banking system, the total money involved in a transfer should remain consistent. Database constraints, such as account balance rules, must not be violated.

I – Isolation:

Isolation ensures that multiple transactions executing simultaneously do not interfere with each other.

Example: If two users try to book the last available airline seat at the same time, isolation prevents both transactions from successfully booking the same seat.

D – Durability:

Durability ensures that once a transaction is committed, its changes remain permanently stored even if a system crash occurs.

Example: After an online shopping order is confirmed and payment is committed, the order details must not disappear because of a power failure.

Thus, ACID properties are critical in systems such as banking, e-commerce, railway reservations, and hospital databases.

5. Concurrency control is used to manage multiple transactions executing at the same time.

1. Dirty Read:

A dirty read occurs when one transaction reads data written by another transaction before that transaction commits.

Example: Transaction T1 changes an account balance from ₹10,000 to ₹8,000 but has not committed. Transaction T2 reads ₹8,000. If T1 later aborts, T2 has used invalid data.

2. Lost Update:

A lost update occurs when two transactions read and update the same data simultaneously, causing one update to overwrite the other.

Example: T1 and T2 both read a stock quantity of 100. T1 changes it to 90, while T2 changes it to 80. If T2's update overwrites T1's update, T1's change is lost.

3. Unrepeatable Read:

An unrepeatable read occurs when a transaction reads the same data twice but receives different values because another transaction modified and committed the data between the two reads.

Example: T1 reads an employee's salary as ₹50,000. T2 updates and commits the salary as ₹55,000. When T1 reads the salary again, it gets a different value.

How 2PL solves these problems

In Two-Phase Locking (2PL), transactions use locks before accessing data.

- A shared lock (S-lock) is used for reading.
- An exclusive lock (X-lock) is used for writing.

A transaction has two phases:

1. Growing Phase: It can acquire locks but cannot release them.
2. Shrinking Phase: It can release locks but cannot acquire new locks.

2PL prevents conflicting operations from occurring at the same time and helps produce conflict-serializable schedules. Strict 2PL, in which exclusive locks are held until commit or abort, also prevents dirty reads and simplifies recovery.

6. A deadlock occurs when two or more transactions wait indefinitely for each other to release resources.

For example:

- T1 holds Lock A and waits for Lock B.
- T2 holds Lock B and waits for Lock A.

Neither transaction can continue, so a deadlock occurs.

Deadlock Detection Using a Wait-For Graph

A Wait-For Graph (WFG) is used for deadlock detection.

- Each transaction is represented as a node.
- An edge from T1 to T2 means T1 is waiting for a resource held by T2.

The algorithm is:

1. Create a node for every active transaction.
2. Add an edge $T1 \rightarrow T2$ if T1 is waiting for a resource held by T2.
3. Search the graph for a cycle.
4. If a cycle exists, a deadlock is present.
5. Select one transaction as a victim.
6. Abort or roll back the victim transaction.
7. Release its resources and allow other transactions to continue.

For example, if the graph contains:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

a cycle exists, so the transactions are deadlocked.

Deadlock Prevention Strategies

1. Resource Ordering:

Assign an order to resources and require every transaction to request resources in the same order. This prevents circular waiting.

2. Wait-Die Scheme:

Transactions are assigned timestamps. An older transaction may wait for a younger transaction, but a younger transaction requesting a resource held by an older transaction is aborted and restarted later.

3. Wound-Wait Scheme:

An older transaction can force a younger transaction to abort if it needs the resource. A younger transaction waits when the resource is held by an older transaction.

At least two commonly used strategies are resource ordering and timestamp-based schemes such as wait-die or wound-wait.

7. Database recovery techniques are used to restore the database after a transaction failure or system crash.

Immediate Update

In the Immediate Update technique, changes made by a transaction may be written to the database before the transaction commits. Therefore, the recovery system may need both:

- **UNDO:** Remove changes made by uncommitted transactions.
- **REDO:** Reapply changes made by committed transactions that were not fully written to disk.

The log normally stores both old and new values.

Deferred Update

In the Deferred Update technique, updates are not applied permanently to the database until the transaction commits. Therefore, uncommitted transactions do not modify the database.

Recovery mainly requires:

- REDO for committed transactions whose changes need to be reapplied.
- No UNDO for uncommitted transactions.

Comparison

Immediate Update

Updates may reach the database before commit

Requires UNDO and REDO

More complex recovery

Allows earlier writing of updates

Deferred Update

Updates are applied after commit

Mainly requires REDO

Simpler recovery

Updates are delayed until commit

When preferred:

Immediate Update is preferred in high-performance database systems where changes may need to be written before commit and sophisticated logging and recovery mechanisms are available.

Deferred Update is preferred in simpler systems where easier recovery is important and delaying updates until commit is acceptable.

8. Shadow Paging is a database recovery technique that uses two page tables:

1. Shadow Page Table: Represents the stable, consistent version of the database.
2. Current Page Table: Used by the active transaction for modifications.

Initially, both tables point to the same database pages. When a transaction modifies a page, the original page is not overwritten. Instead, the modified data is written to a new page, and the current page table is updated to point to the new page.

When the transaction commits, the current page table becomes the new shadow page table. If the transaction fails, the modified pages are discarded and the original shadow page table is retained.

Advantages

- Recovery is simple because the original database pages remain unchanged.
- No complex UNDO operation is required.
- Crash recovery can be fast.
- The database always has a consistent shadow version.

Limitations

- Maintaining page tables can consume additional storage.

- Copying page tables can create overhead.
- It may cause disk fragmentation because updated pages are stored in different locations.
- It is difficult to manage efficiently when many transactions execute concurrently.
- It is less suitable for large, highly concurrent database systems.

Comparison with Log-Based Recovery

Log-based recovery uses transaction logs to record database modifications and performs UNDO and/or REDO operations after a crash. It generally provides better support for large-scale concurrent systems.

Shadow paging provides simpler recovery, whereas log-based recovery is more flexible and scalable. Therefore, modern DBMSs generally prefer advanced log-based recovery techniques for large transactional systems.

9. A schedule is the order in which operations of multiple transactions are executed.

A schedule is called serializable if its final result is equivalent to some serial execution of the same transactions. In a serial schedule, one transaction completes before the next transaction begins.

Conflict serializability is tested using a precedence graph, also called a serialization graph.

Steps to Test Conflict Serializability

Step 1: Create a node for each transaction.

For example, if there are transactions T1, T2, and T3, create three nodes.

Step 2: Identify conflicting operations.

Two operations conflict when:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

Examples of conflicts are:

- Read-Write conflict

- Write-Read conflict
- Write-Write conflict

Step 3: Add directed edges.

If an operation of T1 occurs before a conflicting operation of T2, add an edge:

$T1 \rightarrow T2$

Step 4: Check for a cycle.

- If the precedence graph has no cycle, the schedule is conflict serializable.
- If the graph contains a cycle, the schedule is not conflict serializable.

For example:

$T1 \rightarrow T2 \rightarrow T3$

There is no cycle, so the schedule is conflict serializable. The equivalent serial order is T1, T2, T3.

However:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

contains a cycle. Therefore, the schedule is not conflict serializable.

10. Two-Phase Locking (2PL) is a concurrency control protocol that uses locks to ensure correct execution of transactions.

1. Basic 2PL

Basic 2PL has two phases:

Growing Phase:

The transaction can acquire locks but cannot release any lock.

Shrinking Phase:

The transaction can release locks but cannot acquire new locks.

Basic 2PL guarantees conflict serializability. However, it may allow cascading rollbacks and can cause deadlocks.

2. Strict 2PL

In Strict 2PL, transactions hold all exclusive (write) locks until they commit or abort.

This prevents other transactions from reading uncommitted data. Therefore, Strict 2PL prevents dirty reads and cascading rollbacks and makes database recovery easier.

3. Rigorous 2PL

In Rigorous 2PL, a transaction holds both shared and exclusive locks until it commits or aborts.

This is stricter than Strict 2PL and provides a serial order corresponding to the order in which transactions commit.

Comparison

Protocol	Locks Held Until	Main Advantage
Basic 2PL	Locks may be released during shrinking phase	Guarantees conflict serializability
Strict 2PL	Exclusive locks held until commit/abort	Prevents cascading rollback and dirty reads
Rigorous 2PL	All shared and exclusive locks held until commit/abort	Provides stronger isolation

Most Commonly Used

Strict 2PL is the most commonly used variation in traditional database systems. It provides a good balance between concurrency and safety. It guarantees conflict serializability, prevents transactions from reading uncommitted changes, avoids cascading rollbacks, and simplifies recovery.

Although deadlocks can still occur under Strict 2PL, they can be handled using deadlock detection, prevention, or timeout mechanisms.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and

		attitude, and professional behavior throughout			professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately